



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

Guia Completo: Migração de JavaScript para TypeScript (com códigos)

Este documento apresenta um exemplo prático de migração de um projeto simples em JavaScript para TypeScript, incluindo passos, configurações e códigos completos para os estudantes.

1) Projeto inicial em JavaScript

Estrutura de pastas sugerida:

```
ts-migracao-demo/  
├─ index.html  
└─ src/  
    └─ main.js
```

Arquivo HTML inicial:

index.html (JS)

```
<!DOCTYPE html>  
<html lang="pt-BR">  
<head>  
  <meta charset="utf-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1" />  
  <title>Migração JS → TS</title>  
</head>  
<body>  
  <h1>Migração de JavaScript para TypeScript</h1>  
  
  <input id="filtro" placeholder="Filtrar habilidades..." />  
  <button id="btn">Filtrar</button>  
  <ul id="lista"></ul>  
  
  <script type="module" src="./src/main.js"></script>  
</body>  
</html>
```



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

src/main.js

```
// dados sem tipos
const skills = [
  { nome: "HTML", nivel: 85 },
  { nome: "CSS", nivel: 75 },
  { nome: "JavaScript", nivel: 70 },
  { nome: "TypeScript", nivel: 55 },
];

function render(lista) {
  const ul = document.getElementById("lista");
  ul.innerHTML = "";
  lista.forEach((s) => {
    const li = document.createElement("li");
    li.textContent = `${s.nome} - ${s.nivel}`;
    ul.appendChild(li);
  });
}

function filtrar(termo) {
  return skills.filter((s) =>
    s.nome.toLowerCase().includes(String(termo || "").toLowerCase())
  );
}

document.getElementById("btn").addEventListener("click", () => {
  const termo = document.getElementById("filtro").value;
  render(filtrar(termo));
});

// primeira renderização
render(skills);
```

2) Passos para migrar para TypeScript

Passo 0 (opcional) — Inicializar npm:

```
npm init -y
```

Passo 1 — Instalar TypeScript:

```
npm i -D typescript
```



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

Passo 2 — Criar tsconfig.json:

```
npx tsc --init
```

Recomendação: editar o tsconfig.json para ativar modo estrito e organizar pastas:

tsconfig.json (recomendado)

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
    "moduleResolution": "bundler",
    "strict": true,
    "noImplicitAny": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src"]
}
```

Passo 3 — Renomear arquivos .js para .ts e ajustar a referência no HTML para apontar para o JS compilado:

index.html (após TS)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Migração JS → TS</title>
</head>
<body>
  <h1>Migração de JavaScript para TypeScript</h1>

  <input id="filtro" placeholder="Filtrar habilidades..." />
  <button id="btn">Filtrar</button>
  <ul id="lista"></ul>

  <!-- agora apontamos para o JS gerado em dist -->
  <script type="module" src="./dist/main.js"></script>
```



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

```
</body>
```

```
</html>
```

3) Código migrado para TypeScript

src/main.ts

```
// Tipos e interfaces
interface Skill {
  nome: string;
  nivel: number; // 0-100
}

const skills: Skill[] = [
  { nome: "HTML", nivel: 85 },
  { nome: "CSS", nivel: 75 },
  { nome: "JavaScript", nivel: 70 },
  { nome: "TypeScript", nivel: 55 },
];

// elementos do DOM com tipos adequados
const ul = document.getElementById("lista") as HTMLUListElement | null;
const input = document.getElementById("filtro") as HTMLInputElement | null;
const btn = document.getElementById("btn") as HTMLButtonElement | null;

// função pura com tipos nos parâmetros e retorno
function filtrar(termo: string, base: Skill[]): Skill[] {
  const t = termo.trim().toLowerCase();
  return base.filter((s) => s.nome.toLowerCase().includes(t));
}

function render(lista: Skill[]): void {
  if (!ul) return;
  ul.innerHTML = "";
  for (const s of lista) {
    const li = document.createElement("li");
    li.textContent = `${s.nome} - ${s.nivel}`;
    ul.appendChild(li);
  }
}
```



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

```
btn?.addEventListener("click", () => {  
  const termo = input?.value ?? "";  
  render(filtrar(termo, skills));  
});
```

```
// primeira renderização  
render(skills);
```

4) Compilar e automatizar (build/watch)

Compilar manualmente:

```
npx tsc
```

Sugestão de scripts no package.json:

package.json (trecho)

```
{  
  "name": "ts-migracao-demo",  
  "private": true,  
  "scripts": {  
    "build": "tsc",  
    "watch": "tsc -w",  
    "serve": "npx live-server --open=./index.html"  
  },  
  "devDependencies": {  
    "typescript": "^5.5.0"  
  }  
}
```

5) Antes x Depois — diferenças importantes

Antes (JS): parâmetros sem tipo, menos autocompletar e verificação frouxa.

```
function filtrar(termo) {  
  return skills.filter((s) =>  
    s.nome.toLowerCase().includes(String(termo || "").toLowerCase())  
  );  
}
```

Depois (TS): interface, parâmetros e retorno tipados, segurança e editor mais inteligente.



UNIVERSIDADE DO OESTE DE SANTA CATARINA

Área: Ciências Exatas e Tecnológicas

Curso: Ciência da Computação

Disciplina: Programação IV

Professor: Roberson Junior Fernandes Alves

Carga horária: 40 h/a

Semestre Letivo: 2025/02

```
function filtrar(termo: string, base: Skill[]): Skill[] {  
    const t = termo.trim().toLowerCase();  
    return base.filter((s) => s.nome.toLowerCase().includes(t));  
}
```

6) Checklist de migração

- [x] Instalar TypeScript
- [x] Criar tsconfig.json
- [x] Ativar strict: true
- [x] Organizar pastas src → dist
- [x] Renomear .js para .ts gradualmente
- [x] Tipar dados, funções e elementos do DOM
- [x] Configurar build/watch
- [x] (Opcional) ESLint + Prettier para TS

7) Vantagens práticas do TypeScript

- Detecção precoce de erros (null/undefined, parâmetros errados).
- Autocompletar (IntelliSense) muito mais preciso.
- Refatoração segura (renomear campos/métodos sem riscos ocultos).
- Código auto-documentado via interfaces e tipos.
- Escalabilidade e manutenção melhores em times e projetos grandes.